

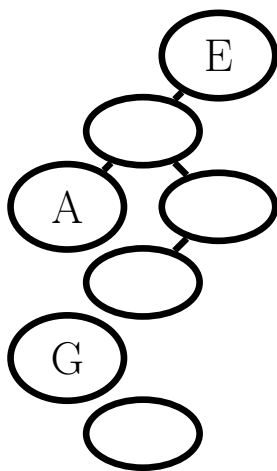


Università degli Studi di Ferrara

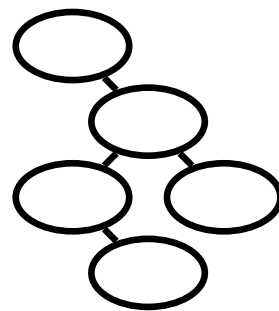
UNIVERSITÀ DEGLI STUDI DI FERRARA

CORSO DI LAUREA IN INFORMATICA

Random Forest



*su immagini
satellitari per
il
riconoscimento
e il conteggio
di piante*



Relatore:

Prof. Guido SCIAVICCO

Laureando:

Tobia SACCHETTO

ANNO ACCADEMICO 2025 – 2026

Indice

	Page
1 Introduzione	5
2 Conoscenze Preliminari	7
2.1 Cosa è l'analisi delle immagini satellitari	8
2.1.1 Dati di una fotografia satellitare	8
2.2 Introduzione al Machine Learning	9
2.2.1 Tipologie di apprendimento	9
2.2.2 Apprendimento Simbolico e Non Simbolico	13
2.2.3 Bontà del modello	14
2.2.4 Alberi Decisionali	15
2.2.5 Entropia	16
2.2.6 Guadagno Informativo	19
2.2.7 Foresta Casuale	19
2.2.8 Regressione Lineare	20
2.2.9 Normalizzazione e filtratura di un dataset	21
2.3 Metodi noti per la risoluzione del problema	24
3 Creazione del modello	27
3.1 Valutazione dei dataset	28
3.2 Valutazione degli approcci possibili	30

3.3	Valutazione degli approcci possibili	30
-----	--	----

Introduzione

In vasti terreni contare il numero di piante specifiche e identificarle è un problema. Basti pensare ad una piccola azienda agricola con qualche ettaro di terreno e qualche migliaia di piante. Lo studio proposto in questa tesi espone un metodo per identificare e contare il numero di piante di avocado in un terreno aziendale tra la Spagna e il Portogallo attraverso tecniche di machine learning basate su immagini satellitari.

Lo scopo è fornire un algoritmo che attraverso un immagine mi identifichi il numero di piante, e anche di conseguenza la dimensione totale della piantagione, per una rapida verifica del controllo del territorio agricolo.

Conoscenze Preliminari

In questo capitolo affronteremo le basi per comprendere successivamente appieno la struttura e i passaggi del lavoro. Il seguente capitolo è suddiviso in 2 parti:

- La prima in cui si spiega che cosa è l'analisi di immagini satellitari, il tipologia delle immagini su cui si è lavorato e come è possibile riconoscere.
- La seconda sezione introdurrà invece i concetti fondamentali del machine learning, con focus specifico sul machine learning simbolico che useremo nel progetto.

2.1 Cosa è l'analisi delle immagini satellitari

L'analisi delle immagini satellitari si riferisce al processo di estrazione di informazioni importanti dalle fotografie aeree scattate da satelliti orbitali. Questo campo combina la disciplina del telerilevamento con tecniche di apprendimento automatico (computer vision) per interpretare grandi quantità di dati visivi.

2.1.1 Dati di una fotografia satellitare

All'interno di una fotografia comune (anche quella scattata dai telefoni), è possibile ricavarne i colori RGB che stanno per Red (Rosso), Green(Verde) e Blue(Blu). attraverso la combinazione di questi 3 valori si può ricreare tutti i colori dello spettro visibile (osservabile con gli occhi). Tuttavia, nelle immagini satellitari, vi sono anche i canali infrarosso quali RedEdge, NIR e SWIR. Si mostra una tabella 2.1 di alcune frequenze d'onda della radiazione elettromagnetica. Ad ogni frequenza di questa appartiene una categoria. Si mostra anche un'immagine 2.1 che mostra lo spettro delle luce visibile e dell'infrarosso.

Tipologia	Lunghezza d'onda
Viola	$\sim 380 - 450nm$
Blu	$\sim 450 - 495nm$
Verde	$\sim 495 - 570nm$
Giallo	$\sim 570 - 590nm$
Arancione	$\sim 590 - 620nm$
Rosso	$\sim 620 - 705nm$
RedEdge	$\sim 705 - 740nm$
NIR	$\sim 760 - 900nm$
SWIR	$\sim 1150 - 2350nm$

Tabella 2.1: tabella infrarosso

Attraverso l'infrarosso si può comprendere caratteristiche invisibili all'occhio umano per esempio lo stato di salute della vegetazione o i livelli di umidità del suolo.

Per lavorare con questi tipi di immagini è stato usato il software open source

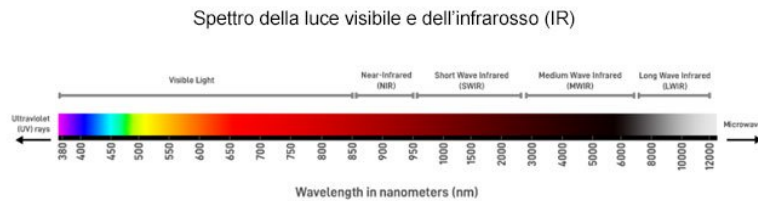


Figura 2.1: spettro della radiazione visibile e dell'infrarosso

QGIS. Queste immagini hanno un formato `.tiff` e i file con cui è stato lavorato sono chiamati `...6band.tiff` dove, in questo caso, `6band` sta per *Red + Green + Blue + RedEdge + NIR + SWIR*. Un esempio di un'immagine satellitare, con `bbox` (bounding box) disegnate per indicare la pianta, è riportata 2.2. Un'immagine è un insieme di pixel con coordinate (x, y) dove x mi indica la posizione del pixel nell'immagine sull'asse delle ascisse (asse orizzontale) e y sull'asse delle ordinate (asse verticale). Ogni pixel, oltre alle coordinate di riferimento, ha un valore che mi indica l'intensità del rosso, uno del verde, uno del blu, uno del RedEdge, uno del NIR e uno del canale SWIR.

2.2 Introduzione al Machine Learning

Il machine learning è il sottoinsieme dell'intelligenza artificiale (AI) basato su algoritmi in grado di "apprendere" i modelli dei dati di addestramento e, successivamente, fare previsioni accurate sui nuovi dati. Questa capacità di riconoscimento dei modelli consente ai modelli di machine learning di prendere decisioni o fare previsioni senza istruzioni esplicite e codificate.

Si definisce dati di addestramento o training data una parte di dati di un dataset che verrà usata per allenare il modello. Il dataset viene definito come un insieme di dati organizzati ($X = x_1, \dots, x_n$), dove a ogni elemento facente parte di esso, detto istanza o sample, rappresenta una misurazione di un certo fenomeno di interesse. Un esempio di un dataset potrebbe essere quello di una farmacia

2.2.1 Tipologie di apprendimento

Possiamo distinguere i modelli in 3 principali tipi di apprendimento automatico:



Figura 2.2: Immagine satellitare con identificazione piante

Esempio	Farmaco	Categoria	Prezzo	Disponibilità
x_1	Tachipirina 1000 mg	Analgesico	7.50	Disponibile
x_2	Aspirina 500 mg	Antinfiammatorio	5.20	Disponibile
x_3	Moment 200 mg	Antidolorifico	6.80	Esaurito
x_4	Augmentin 875 mg	Antibiotico	12.40	Disponibile
x_5	Enterogermina	Probiotico	9.90	Disponibile
x_6	Benagol Gola	Pastiglie gola	4.30	Disponibile
x_7	Voltaren Gel	Antinfiammatorio	10.50	Esaurito
x_8	Zyrtec 10 mg	Antistaminico	8.70	Disponibile

Tabella 2.2: Esempio di dataset relativo ai prodotti di una farmacia

- **Apprendimento Supervisionato:**

Nel dataset oltre all'input x_i viene associato (ad ogni input x_i) un output y_i . Si riporta una tabella esempio come la precedente. In questo caso y_i indica se deve essere comprato oppure no il farmaco dal grossista. Il modello

Input	Output: Comprare?
x_1	$y_1 = SI$
x_2	$y_2 = NO$
x_3	$y_3 = NO$
x_4	$y_4 = SI$
x_5	$y_5 = SI$
x_6	$y_6 = NO$
x_7	$y_7 = SI$
x_8	$y_8 = SI$

Tabella 2.3: Esempio di coppie di un dataset

osserva coppie di input-output $((x_1, y_1), \dots, (x_n, y_n))$ e apprende una funzione (chiamata $\hat{h}$) che dato l'input (x_i) , il valore della funzione $\hat{h}(x)$ cerca di farlo avvicinare all'output corretto (y_i) . Per esempio, gli input potrebbero essere immagini ottenute da una fotocamera, ognuna accompagnata da un output che indica “bus” o “pedone” e così via. Un output come questo è chiamato etichetta. Nel nostro esempio l'output è se il farmacista dovrebbe comprarlo dal grossista oppure no, mentre le etichette sono Farmaco, Categoria, Prezzo

e Disponibilità Una funzione è dato un input x_i si ricava il risultato. Per esempio la funzione

$$h(x_i) = y_i \quad (2.1)$$

$$h(x_i) = 2x_i + 3 \quad (2.2)$$

$$h(3) = 2 \cdot 3 + 3 = 9 \quad (2.3)$$

Tuttavia, nel machine learning la forma della funzione $(2x + 3)$ non viene fissata in anticipo: non la conosciamo. Questa è calcolata automaticamente a partire dai dati di addestramento.

- **Apprendimento Non Supervisionato:**

Il modello apprende i pattern dall'input (x_i) senza alcun feedback esplicito (y_i). L'attività di apprendimento non supervisionato più comune è il clustering: individuare cluster potenzialmente utili di esempi di input. Per esempio, un sistema di visione artificiale, osservando milioni di immagini prese da Internet, è in grado di identificare un ampio cluster di immagini simili che una persona chiamerebbe gatti. Per esempio nell'immagine sottostante i puntini in rosso sono i gatti, in verde le piante e in blu i pesci.

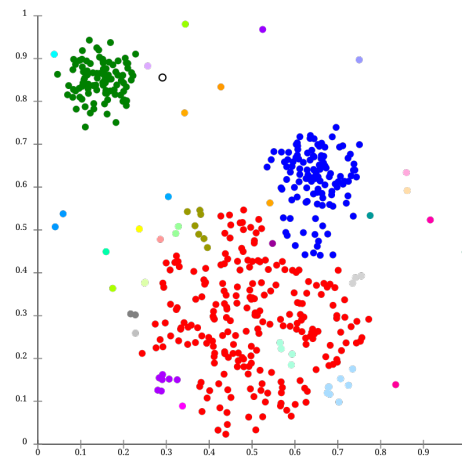


Figura 2.3: Rappresentazione di un cluster di un dataset

- **Apprendimento Rinforzato:**

L'apprendimento avviene grazie ad una serie di rinforzi: il modello impara da risposte positive o negative sul risultato. Ad esempio, dopo una partita a un gioco scelto, possono essere assegnati premi per la vittoria o punizioni per la sconfitta in base a come ha giocato il modello. Il gioco viene ripetuto e le azioni e scelte prese vengono progressivamente alterate dal modello in modo che questo ricerchi una ricompensa.

In questa tesi si andrà a lavorare con la tipologia di apprendimento supervisionato.

2.2.2 Apprendimento Simbolico e Non Simbolico

Il machine learning simbolico è un approccio all'apprendimento automatico che si basa sulla rappresentazione esplicita della conoscenza, tramite simboli (cioè segni che indicano entità, categorie e relazioni che vogliamo rappresentare (ad esempio `gatto`, `essere vivente`, `appartiene a`)), regole logiche espresse tipicamente come implicazioni (se succede questo, allora...) e inferenza (ragionamento deduttivo).

Un esempio di ragionamento deduttivo è il seguente: a partire da regole generali e da fatti specifici, una macchina deduce una conclusione logicamente necessaria (se è vero che `tutti i mammiferi allattano` e che `Fido è un mammifero`, allora deduco che `Fido allatta`).

L'apprendimento simbolico non necessita di grandi quantità di dati per l'addestramento, ha un'alta trasparenza nelle decisioni (perché ogni ragionamento si può comprendere attraverso regole logiche) e ha limiti nella gestione di dati rumorosi o ambigui.

L'apprendimento simbolico si contrappone all'apprendimento statistico (ad esempio le reti neurali), basato su correlazioni numeriche estratte dai dati. Quest'ultimo ha bisogno di grandi quantità di dati nell'apprendimento, sono delle blackbox (dato un input si ha un output, ma non si conosce il ragionamento dietro) e è buono nella gestione di dati rumorosi o ambigui.

2.2.3 Bontà del modello

Per valutare se il modello è buono per il problema si neccessità di conoscere i seguenti parametri.

- F1-macro
- ROC-AUC
- Prec-macro
- Rec-macro
- Positivi Veri (PV_i): pixel correttamente predetti come positivi;
- Negativi Veri (NV_i): pixel correttamente predetti come negativi;
- Positivi Falsi (PF_i) - pixel negativi ma predetti positivi;
- Negativi Falsi (NF_i) - pixel positivi ma predetti negativi.

Da qui possiamo calcolare:

- $Precision_i$:proporzione dei pixel classificati come piante e che rappresentano effettivamente delle piante.

$$Precision_i = \frac{PV_i}{PV_i + PF_i}$$

- $Recall_i$: calcola la percentuale di obiettivi da riconoscere che sono stati riconosciuti.

$$Recall_i = \frac{PV_i}{PV_i + NF_i}$$

- $F1_i$:serve per valutare la qualità di un modello

$$F1_i = 2 \cdot \frac{Precision_i \cdot Recall_i}{Precision_i + Recall_i}$$

$$\text{Prec-macro} = \frac{1}{N} \sum_{i=1}^N \text{Precision}_i \quad (2.4)$$

$$\text{Rec-macro} = \frac{1}{N} \sum_{i=1}^N \text{Recall}_i \quad (2.5)$$

$$\text{F1-macro} = \frac{1}{N} \sum_{i=1}^N \text{F1}_i \quad (2.6)$$

2.2.4 Alberi Decisionali

Possiamo visualizzare la tabella 2.2 e 2.3 come un albero decisionale 2.4.

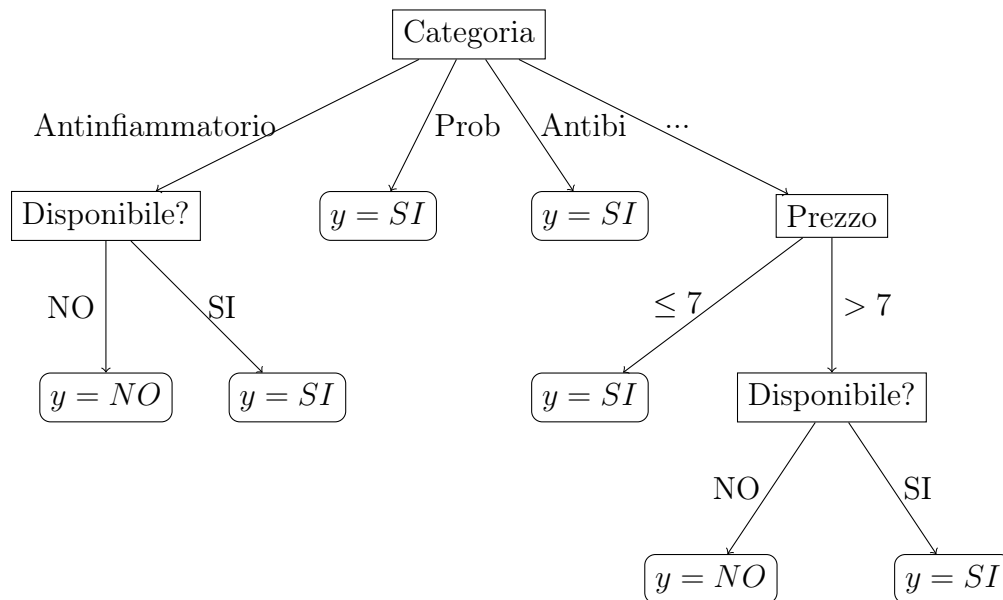


Figura 2.4: Albero decisionale del dataset farmacia

Un albero decisionale serve a risolvere problemi di regressione e di classificazione 2.3. Questo è una rappresentazione di una funzione che dato in input un vettore di valori x_i da un valore di output (una decisione) y_i . Ogni nodo interno all'albero corrisponde ad un attributo della tabella, mentre i nodi esterni (foglie) a delle decisioni ($y = SI$ $y = NO$). I rami etichettati con degli ipotetici valori dell'attributo (Es. *Antinfiammatorio*, *SI*, *NO*,...). Per ... intendiamo tutti gli altri elementi che non fanno parte di quella categoria e le diciture *Prob* e *Antibi* stanno per

rispettivamente Probiotico e Antibiotico. Questo albero in particolare come possiamo osservare viene usato per una classificazione binaria ($y = 0$ o $y = 1$)($y = SI$ $y = NO$), ma possiamo avere più classi ($y = 0, y = 1, y = 2$)

2.2.5 Entropia

Possiamo costruire un gran numero di alberi semplicemente cambiando attributi dell'albero. Per scegliere gli attributi e l'ordine di questi con cui costruire l'albero usiamo il concetto di guadagno di informazione 2.2.6, definito in termini di entropia. L'entropia è la misura dell'incertezza di una variabile casuale.

L'unità di misura dell'entropia in questo caso (dato che stiamo usando un computer (che comunica attraverso 0 o 1)) è il bit.

Entropia di una singola variabile casuale

L'entropia di una variabile casuale discreta V con k elementi è definita come:

$$H(V) = - \sum_k P(v_k) \log_2 P(v_k)$$

Dove v_k sono i possibili valori che la variabile aleatoria V assume. Ognuno di questi valori ha una probabilità $P(v_k) \in [0, 1]$ di accadere. Inoltre $\sum_k P(v_k) = 1$. Per esempio il lancio di una moneta 4 volte. La moneta ha 2 possibilità eque (Testa o Croce). Quindi V sarà un vettore lungo $k = 16$ che contiene tutte i possibili lanci:

$$V = \{TTTT, TTTC, TTCT, \dots, CCCC\} \quad (2.7)$$

$$P(v_k) = \left(\frac{1}{2}\right)^4 \quad (2.8)$$

$$P(V) = \left\{ \left(\frac{1}{2}\right)^4, \left(\frac{1}{2}\right)^4, \dots, \left(\frac{1}{2}\right)^4 \right\} \quad (2.9)$$

Ad esempio

$$P(v_k = TCCT) = \frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{16} \quad (2.10)$$

$$H(V) = - \sum_{k=1}^{k=16} \frac{1}{16} \log_2 \frac{1}{16} = 4\text{bit} \quad (2.11)$$

Quindi per rappresentare un risultato o i risultati dei k elementi della variabile aleatoria V sono necessari 4 bit.

La probabilità di ottenere C o T è $\frac{1}{2}$ e da qui possiamo affermare che la probabilità di ottenere un qualsiasi lancio di moneta 4 volte ($P(v_k)$) è esattamente $(\frac{1}{2})^4$.

Ora per esempio poniamo di avere una moneta truccata che dia T il $\frac{2}{3}$ dei lanci e C il restante $\frac{1}{3}$. Di conseguenza

$$P(v_k = TCTT) = \frac{2}{3} \cdot \frac{1}{3} \cdot \frac{2}{3} \cdot \frac{1}{3} = \frac{8}{81} \quad (2.12)$$

$$P(v_k = TCCT) = \frac{2}{3} \cdot \frac{1}{3} \cdot \frac{2}{3} \cdot \frac{1}{3} = \frac{4}{81} \quad (2.13)$$

$$P(v_k = TCCC) = \frac{2}{3} \cdot \frac{1}{3} \cdot \frac{2}{3} \cdot \frac{1}{3} = \frac{2}{81} \quad (2.14)$$

$$P(v_k = TTTT) = \frac{2}{3} \cdot \frac{1}{3} \cdot \frac{2}{3} \cdot \frac{1}{3} = \frac{16}{81} \quad (2.15)$$

$$P(v_k = CCCC) = \frac{2}{3} \cdot \frac{1}{3} \cdot \frac{2}{3} \cdot \frac{1}{3} = \frac{1}{81} \quad (2.16)$$

$$(2.17)$$

Per la sequenza TCTT, che contiene una sola C, si ottengono in totale 4 configurazioni distinte (ad esempio TCTT,CTTT,TTCT,TTTC). Analogamente, le sequenze con 2 C sono 4, quelle con 3 C sono anch'esse 4, mentre esistono un'unica configurazione con 4 C (CCCC) e una con 0 C (TTTT). Quindi

$$\begin{aligned} H(V) &= - \sum_k P(v_k) \log_2 P(v_k) \\ &= - \left(\left(\frac{8}{81} \log_2 \frac{8}{81} \right) \cdot 4 \right. \\ &\quad + \left(\frac{4}{81} \log_2 \frac{4}{81} \right) \cdot 4 \\ &\quad + \left(\frac{2}{81} \log_2 \frac{2}{81} \right) \cdot 4 \\ &\quad \left. + \frac{16}{81} \log_2 \frac{16}{81} + \frac{1}{81} \log_2 \frac{1}{81} \right) = 3.2446 \end{aligned}$$

Di conseguenza ci servono meno bit di informazione per rappresentare questo caso rispetto il precedente

Entropia Congiunta

Prese 2 variabili aleatorie discrete V e G è l'entropia della loro coppia (V, G) .

$$H(V, G) = - \sum_{v,g} P(v_k, g_k) \log_2 P(v_k, g_k)$$

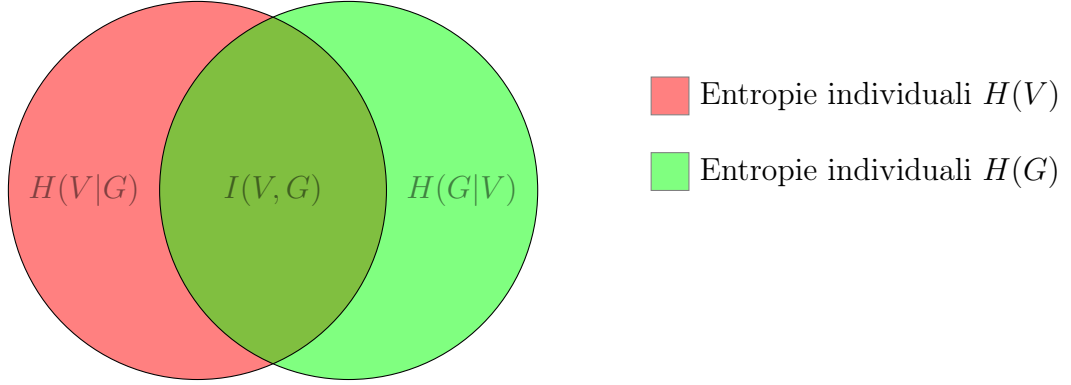


Figura 2.5: Rappresentazione di $H(V, G)$. Questo l'insieme delle entropie delle variabili casuali V e G

Esempio: Consideriamo 2 variabili casuali binarie:

- V esito del primo lancio (Testa o Cionce)
- G esito del secondo lancio (Testa o Croce)

Poniamo che la moneta sia equa e indipendente. Quindi:

$$V = \{T, C\} \quad (2.18)$$

$$G = \{T, C\} \quad (2.19)$$

$$P(V) = P(G) = \left(\frac{1}{2}, \frac{1}{2}\right) \quad (2.20)$$

$$P(v_k, g_k) = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4} \quad (2.21)$$

Dato che la moneta è equa e quindi la probabilità di avere T o C non cambia di conseguenza l'entropia congiunta è:

$$H(V, G) = - \sum_{v,g} P(v_k, g_k) \log_2 P(v_k, g_k) \quad (2.22)$$

$$= - \left(4 \cdot \left(\frac{1}{4} \log_2 \frac{1}{4} \right) \right) = 2\text{bit} \quad (2.23)$$

Entropia Condizionata

L'entropia condizionale $H(V|G)$ è la quantità di informazione necessaria per descrivere il valore di una variabile aleatoria V noto il valore di un'altra variabile aleatoria G . $H(V|G)$ è definita come

$$H(V|G) = \mathbb{E}_G[H(V|g)] = \sum_{v,g} P(v_k, g_k) \log \frac{P(g_k)}{P(v_k, g_k)} \quad (2.24)$$

$$= H(V, G) - H(G) \quad (2.25)$$

Poniamo che Dobbiamo trovare l'entropia $H(V|G)$ dell'esempio precedente:

$$H(V, G) = 2\text{bit} \quad (2.26)$$

$$H(G) = - \left(2 \cdot \left(\frac{1}{2} \log_2 \frac{1}{2} \right) \right) = 1\text{bit} \quad (2.27)$$

$$H(V|G) = H(V, G) - H(G) = 2 - 1 = 1\text{bit} \quad (2.28)$$

2.2.6 Guadagno Informativo

Il guadagno informativo IG serve per capire che attributo(o variabile aleatoria (V o G)) scegliere per un albero decisionale. La formula è:

$$IG(G|V) = H(G) - H(G|V)$$

Per esempio nel caso precedente si avrebbe

$$IG(G|V) = H(G) - H(G|V) = 1 - 1 = 0\text{bit}$$

Quindi il primo lancio V non dice niente del secondo G . Questo non porta informazione. Per la creazione degli alberi viene calcolata l'entropia su tutto il dataset G e poi per ogni attributo disponibile V_i viene calcolato $IG(G|V_i)$ e viene scelto quello con il guadagno più alto.

2.2.7 Foresta Casuale

Una Foresta Aleatoria (o Foresta Casuale) è un insieme di tanti alberi decisionali N . Inoltre questi alberi vengono allenati solo su una parte scelta a caso di tutto il dataset. Questa tecnica viene chiamata bagging. Ogni campione preso in questo

modo viene chiamato campione bootstrap ed viene effettuato con estrazione casuale e reinserimento. Per ogni nodo di questi alberi viene scelto un sottoinsieme di attributi casuali su cui cercare lo split migliore. L'input anziché essere dato a un singolo albero viene dato a tanti alberi in parallelo. I risultati u_i di questi vengono valutati per l'intera Foresta come una media $\left(\frac{1}{N} \cdot \sum_{k=1}^N u_k\right)$ se si tratta di un problema di regressione, oppure come un voto di maggioranza se invece si tratta di un problema di classificazione.

2.2.8 Regressione Lineare

La Regressione Lineare viene definita come un adattamento ad una funzione lineare dei dati (punti).

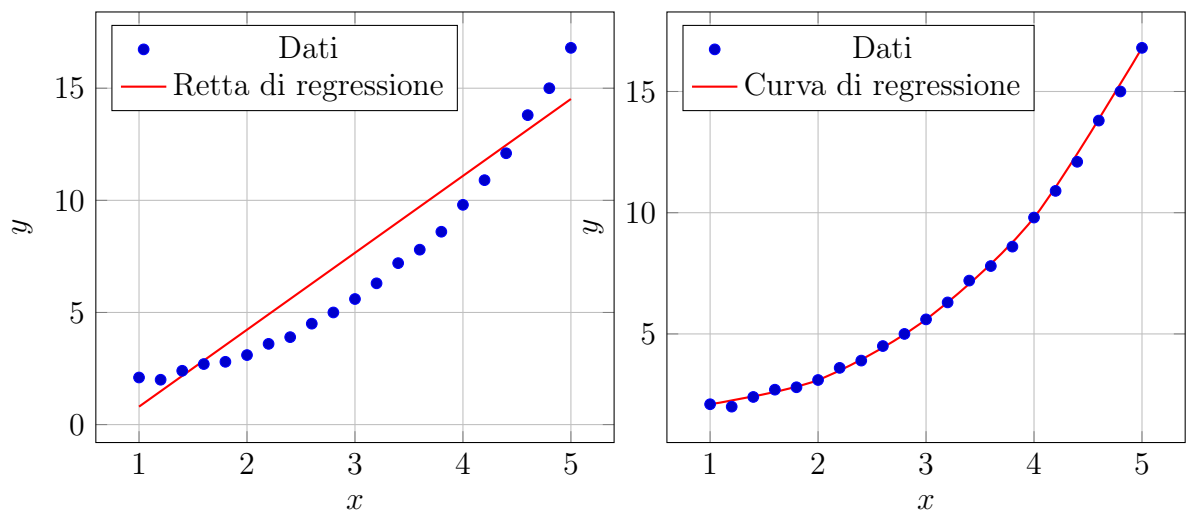


Figura 2.6: A sinistra una Regressione Lineare Univariata (Retta). A destra una approssimazione NON Lineare (Curva lisciata)

Per chiarezza, la curva di regressione è stata creata interpolando solo 5 punti dall'insieme del dataset presi a intervalli regolari. Questo non altera la tendenza poichè la funzione tende a generalizzare bene l'andamento dei punti.

Dati un insieme di i punti con 2 variabili (x, y) , per esempio $(x = \text{Larghezza}, y = \text{Lunghezza})$ la regressione è costruire una funzione che adatta l'andamento dei punti. In questa tesi si affronterà la Regressione Lineare Univariata. Nella Regressione Lineare Univariata la funzione che adatta i punti è una retta. La retta viene

definita tramite la formula del tipo

$$y = mx + q$$

m indica l'inclinazione della retta, mentre q è lo spostamento di quella retta sull'asse delle ordinate. Si vuole trovare la funzione f che approssimi la sequenza dei vari punti. Quindi date i coppie (x_i, y_i) questo problema si risolve utilizzando il Metodo Dei Minimi Quadrati.

$$m = \frac{i \cdot \sum x_i y_i - \sum x_i \cdot \sum y_i}{i \cdot \sum (x_i^2) - (\sum x_i)^2} \quad (2.29)$$

$$q = \frac{\sum y_i \cdot \sum (x_i^2) - \sum x_i \cdot \sum x_i y_i}{i \cdot \sum (x_i^2) - (\sum x_i)^2} \quad (2.30)$$

Di conseguenza si può costruire la precedente funzione f che dà il punto sull'ordinata y_i sapendo solo il punto sulle ascisse x_i .

$$f(x_i) = m \cdot x_i + q = y_i$$

Ulteriori approfondimenti sul Machine Learning si possono affrontare nel libro Russell 2010 o nella sua versione in italiano Russell e Norvig 2022

2.2.9 Normalizzazione e filtratura di un dataset

Per normalizzazione di un dataset si intende dare a un insieme di dati un range garantito. Per esempio si vuole che i Per farlo si usa la seguente formula: dato un insieme di dati X di numero i ,

$$x_i = \frac{x_i - \min(X)}{\max(X) - \min(X)}$$

Dove X è il vettore che contiene tutti gli x_i . Quindi

$$X = \{x_1, x_2, \dots, x_i\}$$

In questa tesi si utilizzeranno dei filtri. Nello specifico filtri di media locale e varianza locale tra i vari pixel. Si mostra una rappresentazione dei vari pixel 2.7. Supponiamo che X sia il vettore che contiene i valori dei pixel rosso.

a_{11}	a_{12}	a_{13}
a_{21}	a_{22}	a_{23}
a_{31}	a_{32}	a_{33}

Figura 2.7: Esempio pixel

Si pone per esempio la seguente griglia (3x3) di visualizzazione dei valori dei pixel di colore rosso e si pongono i seguenti valori:

$$a_{11} = 0.75; a_{12} = 0.75; a_{13} = 0.75; \quad (2.31)$$

$$a_{21} = 0.45; a_{22} = 0.25; a_{23} = 0.95; \quad (2.32)$$

$$a_{31} = 0.35; a_{32} = 0.25; a_{33} = 0.05; \quad (2.33)$$

Di conseguenza nel pixel a_{22} avrò la media effettuata da:

$$media_{3x3rosso}(a_{22}) = \frac{1}{i \cdot j} \cdot \sum_i \sum_j a_{ij} \quad (2.34)$$

$$= \frac{1}{3 \cdot 3} \cdot ((0.75 + 0.75 + 0.75) \quad (2.35)$$

$$+ (0.45 + 0.25 + 0.95) + (0.35 + 0.25 + 0.05)) \quad (2.36)$$

$$= 0.5056 \quad (2.37)$$

Quindi nel layer `media3x3rosso` avremmo nel pixel $a_{22} = 0.5056$. Nel caso si voglia calcolare la `media3x3rosso` nel pixel a_{11} . Per gestire i pixel nei bordi si specchiano i valori nei pixel. Si creano delle caselle virtuali $a_{00}, a_{01}, a_{10}, \dots$ all'interno delle quali verrà inserito il valore del pixel rosso adiacente. Quindi $a_{00} = a_{01} = a_{10} = a_{11} = 0.75$, per $a_{02} = a_{12} = 0.75$, per $a_{20} = a_{21} = 0.45$ e così via... Di conseguenza la media nel pixel a_{11} sarà

$$media_{3 \times 3, rosso}(a_{11}) = \frac{1}{i \cdot j} \cdot \sum_i \sum_j a_{ij} \quad (2.38)$$

$$= \frac{1}{3 \cdot 3} \cdot ((0.75 + 0.75 + 0.75) \text{ che sarebbe } (a_{00}, a_{01}, a_{02}) \quad (2.39)$$

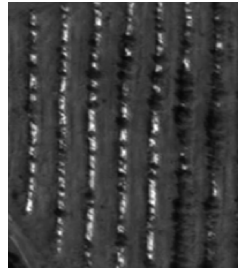
$$+ (0.75 + 0.75 + 0.75) + (0.45 + 0.45 + 0.25)) \quad (2.40)$$

$$= 0.6278 \quad (2.41)$$

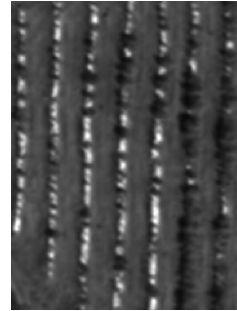
Nella figura la tecnica di fare una media3x3 o 7x7 si rappresenta con una sfocatura 2.10. Si mostrano degli esempi



Figura
Rappresentazione
della immagine
so,verde,blu



2.8: Figura
Rappresentazione della
deviazione standard del
colore rosso in modo
binario (bianco/nero)



2.9: Figura
Rappresentazione della
media3x3 del colore
rosso in modo binario
(bianco/nero)

La deviazione standard per il pixel a_{22} (in questo caso di colore rosso) è data dalla formula

$$\text{devstand}_{3 \times 3, \text{rosso}}(a_{22}) = \sqrt{\frac{1}{i \cdot j} \sum_i \sum_j (x_{ij} - \text{media}_{3 \times 3, \text{rosso}}(a_{22}))^2} \quad (2.42)$$

$$= \sqrt{\frac{1}{3 \cdot 3} \left((0.75 - 0.5056)^2 + (0.75 - 0.5056)^2 + (0.75 - 0.5056)^2 \right. \\ \left. + (0.45 - 0.5056)^2 + (0.25 - 0.5056)^2 + (0.95 - 0.5056)^2 \right. \\ \left. + (0.35 - 0.5056)^2 + (0.25 - 0.5056)^2 + (0.05 - 0.5056)^2 \right)} \quad (2.43)$$

$$= \sqrt{0.0824} \approx 0.2872 \quad (2.44)$$

Anche in questo caso nel caso si voglia calcolare a_{11} i valori vengono specchiati

come nel caso precedente, quindi

$$\text{devstand}_{3 \times 3, \text{rosso}}(a_{11}) = \sqrt{\frac{1}{i \cdot j} \sum_i \sum_j (x_{ij} - \text{media}_{3 \times 3, \text{rosso}}(a_{11}))^2} \quad (2.45)$$

$$= \sqrt{\frac{1}{3 \cdot 3} \left((0.75 - 0.6278)^2 + (0.75 - 0.6278)^2 + (0.75 - 0.6278)^2 \right. \\ \left. (0.75 - 0.6278)^2 + (0.75 - 0.6278)^2 + (0.75 - 0.6278)^2 \right. \\ \left. + (0.45 - 0.6278)^2 + (0.45 - 0.6278)^2 + (0.25 - 0.6278)^2 \right)} \quad (2.46)$$

$$= \sqrt{0.0328} \approx 0.1812 \quad (2.47)$$

La rappresentazione visuale della deviazione standard sul rosso su un'immagine si rappresenta come un accentuamento del cambio di intensità del colore 2.12. Si visualizzano un'immagine per esempio:

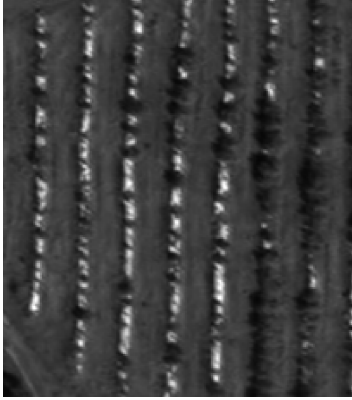


Figura 2.11: Rappresentazione del colore rosso in modo binario (bianco/nero)

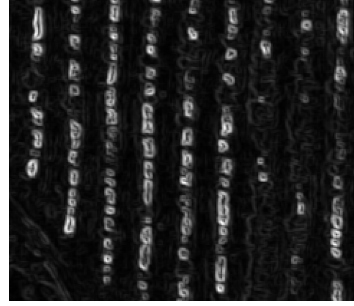


Figura 2.12: Rappresentazione della deviazione standard del colore rosso in modo binario (bianco/nero)

2.3 Metodi noti per la risoluzione del problema

Il problema di identificare le potenziali piante data un'immagine (in questo caso satellitare) non è banale. Non si può dedurre che si riuscirà a identificare la pianta in maniera certa e di conseguenza è necessario procedere a valutare caso per caso

se l'identificazione è possibile.

Alcuni articoli, come Yao et al. 2021, hanno usato una rete neurale (di tipologia CNN) per trovare una mappa di densità degli alberi dall'immagine. Il numero totale degli alberi si ottiene integrando tale mappa. Altri articoli, come Fu et al. 2017, usano una Foresta Casuale 2.2.7 pixel per pixel per mappare la vegetazione delle zone umide.

Creazione del modello

In questo capitolo si andrà ad analizzare i dati e la loro struttura e successivamente si faranno delle scelte riguardo la creazione del modello. infine si mostrerà la creazione del modello stesso usando il lavoro descritto nel capitolo precedente.

3.1 Valutazione dei dataset

Durante la ricerca per trovare dei dataset utilizzabili per l'addestramento del modello oltre a quelli messi a disposizione dell'azienda si è trovato il dataset Faet et al. 2025 che tuttavia non contiene il canale SWIR ed è stato effettuato da un drone e non da un satellite. Per valutare la bontà dei dati e se un ipotetico classificatore è in grado di distinguere i vari pixel si è andati a plottare i valori delle bande. Si mostrano di seguito i plot. Nello specifico si sono create delle bbox di diverse tipologie:

- bbox per indicare le non piante
- bbox per indicare le piante avocado
- bbox per indicare le piante di un'altra tipologia (1)
- bbox per indicare le piante di un'altra tipologia (2)

Le bbox (Bounding Box) sono riquadri di ritaglio nell'immagine su cui si andrà a lavorare. Per ogni bbox di una tipologia si fa il plot dei pixel contenuti all'interno della bbox. Di seguito si mostrano i plot per quanto riguarda le bbox non piante e le bbox che contengono le piante avocado 3.1

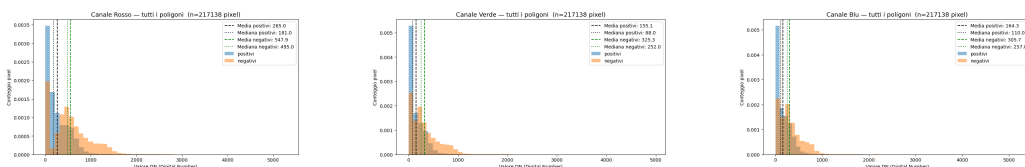


Figura 3.1: Si mostrano i grafici RGB delle bbox degli avocado e dei non pianta

Di seguito si mostrano i plot delle bande dell'infrarosso (RedEdge, NIR, SWIR) 3.2:

Dato che alcuni valori (prossimi allo 0) rendono il grafico poco chiaro e difficile interpretare le differenze tra gli istogrammi, si è deciso di rimuovere questi valori in modo da rendere il grafico interpretabile.

Come si evince dai grafici in quasi tutte le bande gli istogrammi sono quasi tutti sovrapposti. Si riesce a differenziare solo nelle 2 classi (avocado e non pianta) solo nelle bande dei grafici NIR e SWIR. Questa separazione, sebbene non netta,

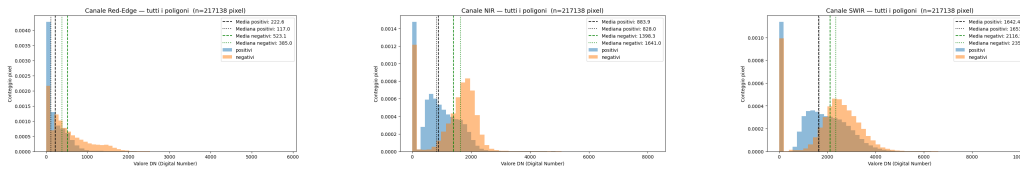


Figura 3.2: Si mostrano i grafici infrarosso delle bbox degli avocado e dei non pianta

indica che le bande infrarosse potrebbero essere le più informative per un ipotetico classificatore.

Si mostrano anche i bbox delle piante di tipologia 1 e 2 NIR prese dalle immagini fornite dall'azienda.

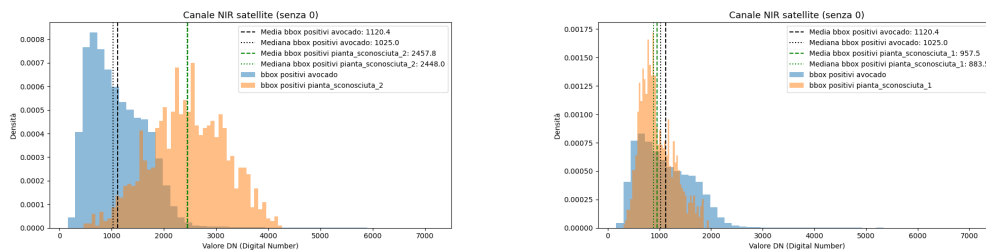


Figura 3.3: Si mostrano i grafici NIR delle bbox delle tipologie di piante 1 e 2

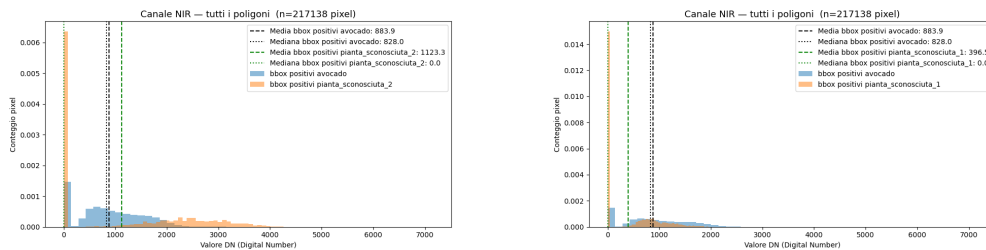


Figura 3.4: Si mostrano i grafici NIR delle bbox delle tipologie di piante 1 e 2 togliendo i valori vicino allo 0

Ora si mostrano i grafici anche per il campo SWIR sempre per le stesse tipologie.

Come si può osservare dai seguenti grafici tutte le tipologie di piante analizzate è possibile distinguere visivamente le distribuzioni di tutte le tipologie.

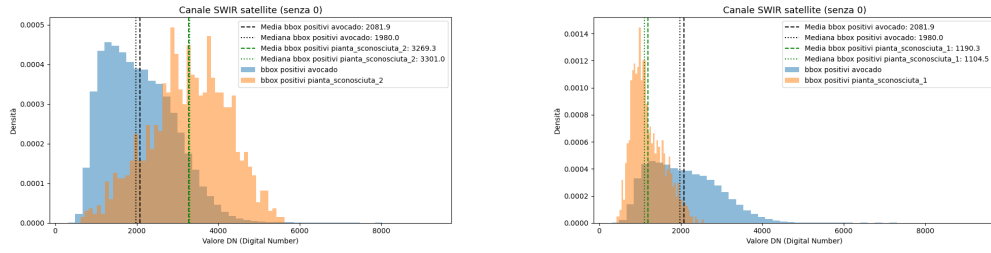


Figura 3.5: Si mostrano i grafici SWIR delle bbox delle tipologie di piante 1 e 2

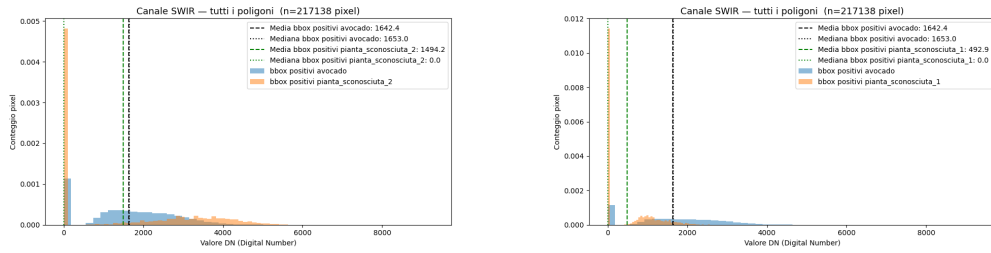


Figura 3.6: Si mostrano i grafici SWIR delle bbox delle tipologie di piante 1 e 2 togliendo i valori vicino allo 0

3.2 Valutazione degli approcci possibili

Per i classificatori si userà una Foresta Casuale 2.2.7 per decidere, per ogni Pixel, se il pixel appartiene oppure no ad una pianta tipologia di pianta. Successivamente si andrà ad applicare una Regressione Lineare 2.2.8 per comprendere il numero di piante dato il numero di pixel di una sagoma.

3.3 Valutazione degli approcci possibili

Si sa che il drone è un DJI Mavic 3 Multispectral. Ha volato con quota 14 m di altezza. La formula per trovare la risoluzione dei pixel è:

$$GSD = \frac{H \cdot S_w}{f \cdot I_w}$$

Dove H è l'altezza in volo, S_w è la larghezza del sensore, f è la focale e I_w è la larghezza immagine in pixel. Quindi la formula diventa:

$$GSD = \frac{14m \cdot 17.3mm}{12mm \cdot 5280px} = 0.38cm/px$$

Di conseguenza:

$$\text{ScaleFactor} = \frac{30\text{cm}/px}{0.38\text{cm}/px} = 78.95$$

Bibliografia

- Faet, Guillem Montalbán et al. (dic. 2025). *Dataset for orange fruit detection from UAV in citrus orchards*. Zenodo. DOI: [10.5281/zenodo.17866344](https://doi.org/10.5281/zenodo.17866344). URL: <https://doi.org/10.5281/zenodo.17866344>.
- Fu, Bolin et al. (2017). «Comparison of object-based and pixel-based Random Forest algorithm for wetland vegetation mapping using high spatial resolution GF-1 and SAR data». In: *Ecological Indicators* 73, pp. 105–117. ISSN: 1470-160X. DOI: <https://doi.org/10.1016/j.ecolind.2016.09.029>. URL: <https://www.sciencedirect.com/science/article/pii/S1470160X1630560X>.
- Russell, Stuart J (2010). *Artificial intelligence a modern approach*. Pearson Education, Inc.
- Russell, Stuart J e Peter Norvig (2022). *Intelligenza artificiale. Un approccio moderno*. Ediz. MyLab. it.
- Yao, Ling et al. (2021). «Tree counting with high spatial-resolution satellite imagery based on deep neural networks». In: *Ecological Indicators* 125, p. 107591. ISSN: 1470-160X. DOI: <https://doi.org/10.1016/j.ecolind.2021.107591>. URL: <https://www.sciencedirect.com/science/article/pii/S1470160X21002569>.